

Patrones de Diseño - IS

Diagrama UML

- Realizar el diagrama UML de clases correspondiente a la implementación.
- Representar las clases, atributos, métodos y relaciones principales del patrón.

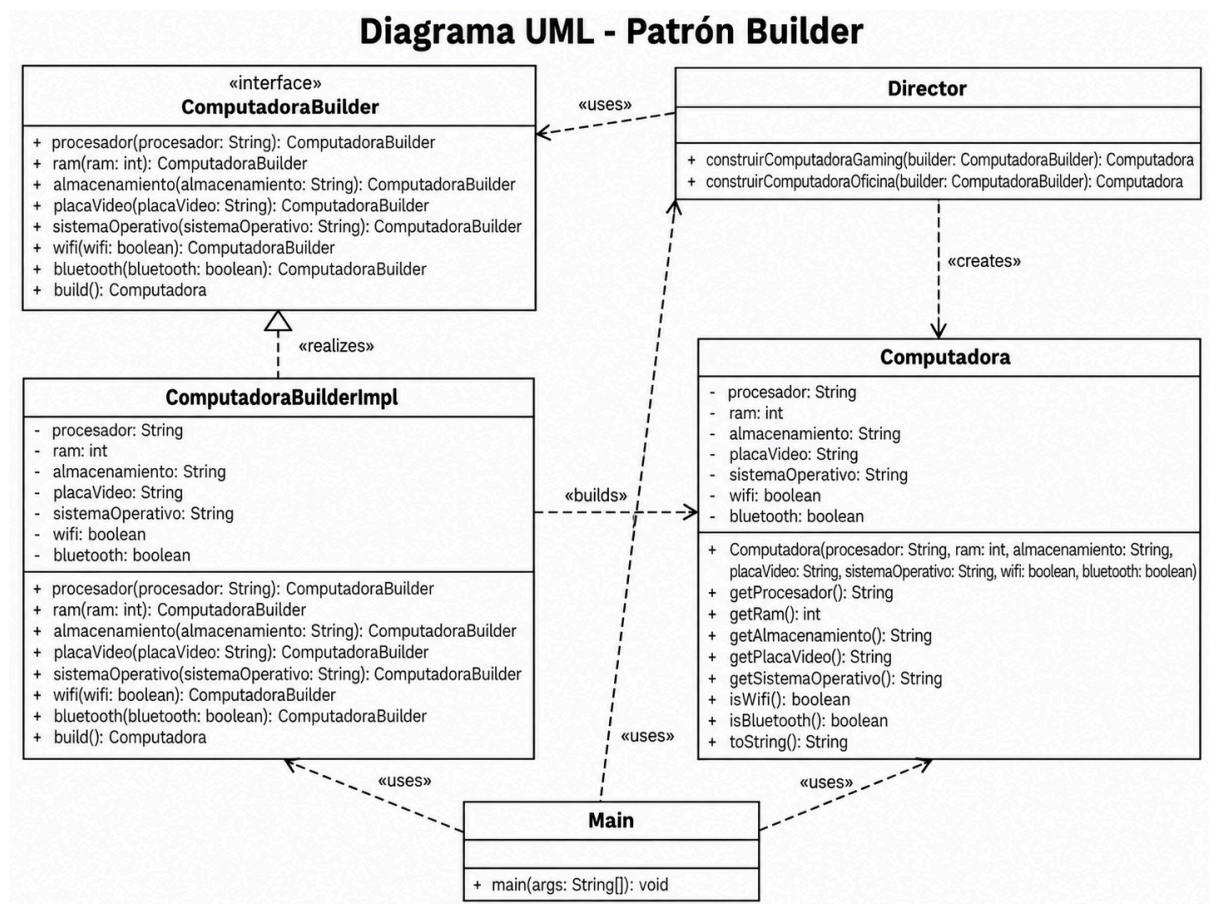
Problema – Solución – Consecuencias

- Explicar qué problema de diseño busca solucionar Builder.
- Explicar cómo el patrón resuelve ese problema.
- Indicar las principales consecuencias, ventajas y posibles desventajas de utilizarlo.

Creational (Creacionales):

Builder

Diagrama UML



En esta implementación participan las siguientes clases:

- **Computadora:** es el producto que se desea construir.
- **ComputadoraBuilder:** interfaz que define los pasos necesarios para configurar una computadora.
- **ComputadoraBuilderImpl:** Builder concreto que implementa los pasos.
- **Director:** define configuraciones predeterminadas de computadoras.
- **Main:** utiliza el Director y el Builder para crear los objetos.

El **ComputadoraBuilder** posee métodos para configurar procesador, RAM, almacenamiento, placa de video, sistema operativo, WiFi y Bluetooth, finalizando con el método `build()`.

Problema

El patrón Builder busca solucionar el problema de crear objetos complejos que poseen muchos atributos o diferentes configuraciones posibles.

Sin este patrón, sería necesario utilizar constructores con una gran cantidad de parámetros.

Solución

Builder separa el proceso de construcción del objeto del objeto final.

En lugar de enviar todos los valores directamente al constructor, se utiliza un **Builder** que permite establecer los atributos paso a paso. Ejemplo:

builder

```
.procesador("Intel i7")  
  
.ram(32)  
  
.almacenamiento("1TB SSD")  
  
.placaVideo("RTX 4070")  
  
.sistemaOperativo("Windows 11")  
  
.wifi(true)  
  
.bluetooth(true)  
  
.build();
```

Consecuencias

Ventajas

- Mayor legibilidad

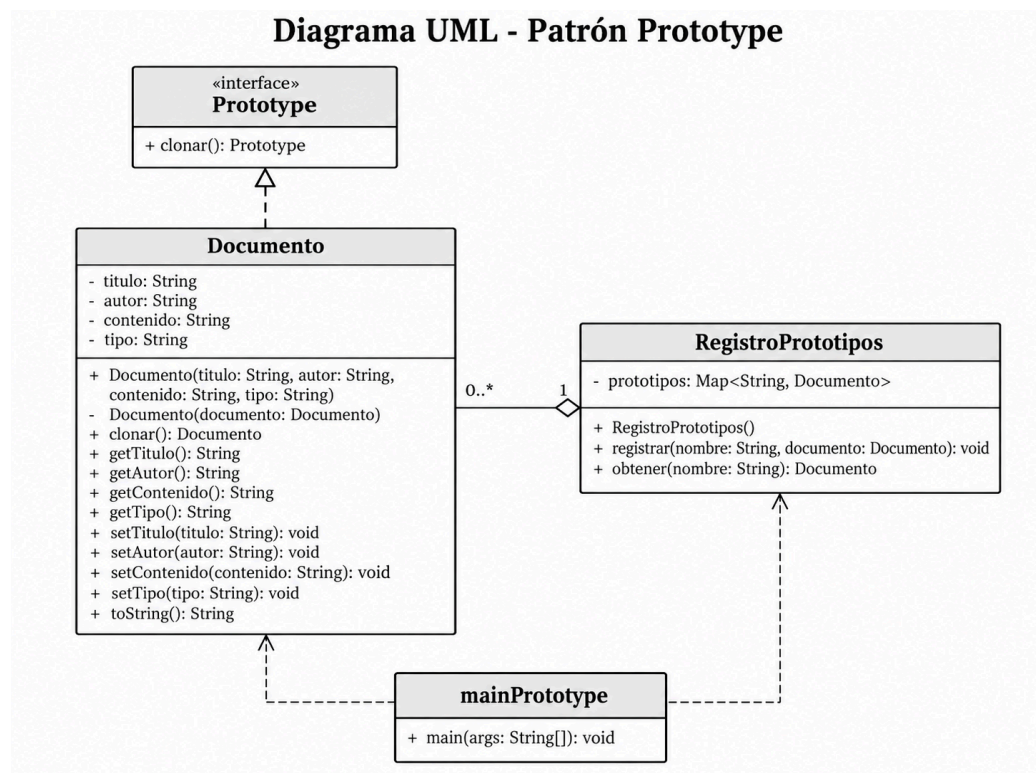
- Construcción paso a paso
- Permite diferentes configuraciones
- Separa la construcción del producto
- Facilita la reutilización
- Permite crear objetos inmutables

Desventajas

- Aumenta la cantidad de clases
- Puede ser innecesario para objetos simples
- Se necesita controlar los objetos obligatorios

Prototype

Diagrama UML



Problema

El patrón Prototype busca solucionar el problema de crear nuevos objetos que sean similares a otros existentes, sin necesidad de construirlos desde cero.

Cuando un objeto tiene muchos atributos o su creación es costosa y compleja, puede resultar ineficiente crear cada instancia manualmente. Además, en muchos casos se necesitan múltiples objetos con una estructura similar, cambiando solo algunos datos específicos.

En este ejemplo, un documento tiene varios atributos como título, autor, contenido y tipo. Crear cada documento desde cero puede ser repetitivo y propenso a errores.

Solución

El patrón **Prototype** resuelve este problema permitiendo crear nuevos objetos a partir de un objeto existente, llamado “**prototipo**”, mediante la clonación.

Cada clase que quiera funcionar como prototipo implementa una interfaz con un método de clonación. En este caso, la interfaz Prototype define **Prototype clonar()**;

La clase Documento implementa este método y devuelve una copia de sí misma utilizando un constructor de copia.

El **RegistroPrototipos** actúa como un repositorio de prototipos, donde se almacenan diferentes documentos modelo que pueden clonar cuando se necesiten nuevas instancias.

Consecuencias

Ventajas

- Evita la creación de objetos desde cero.
- Reduce costos de creación.
- Facilita la creación de múltiples objetos similares.
- Permite una mayor flexibilidad.
- Encapsula el proceso de clonación.

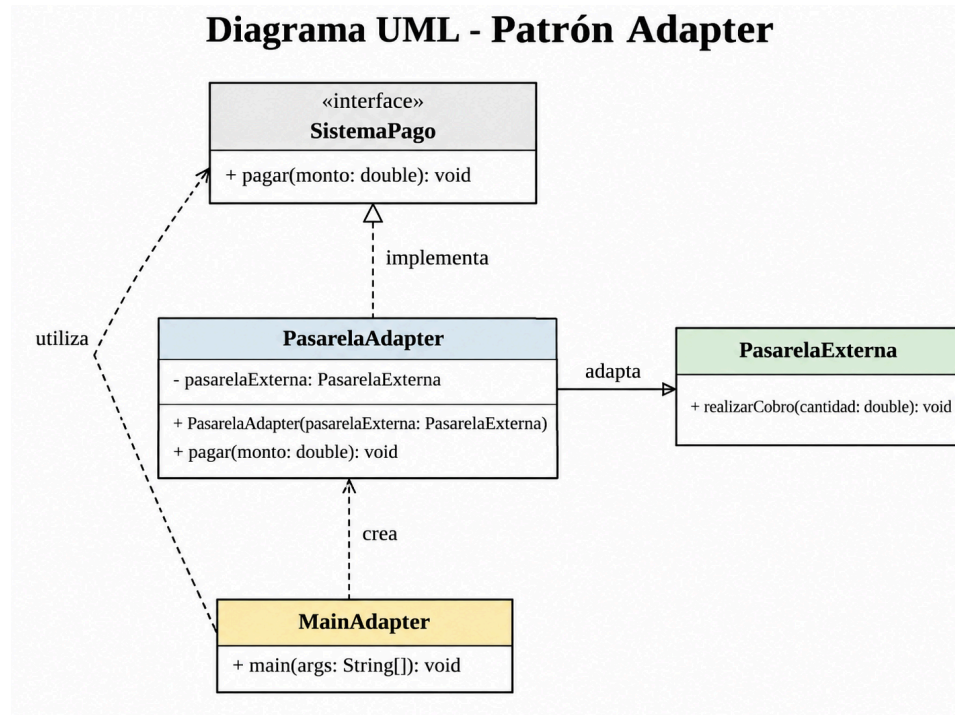
Desventajas

- Puede ser complejo clonar objetos con referencias a otros objetos.
- Requiere que las clases implementen el método de clonación.
- No siempre es la mejor opción.

Structural (Estructurales):

Adapter

Diagrama UML



Problema

El patrón **Adapter** busca solucionar el problema de incompatibilidad entre dos interfaces que necesitan trabajar juntas.

Puede ocurrir que tengamos una clase existente que realiza la función que necesitamos, pero sus métodos o su interfaz son diferentes a los que espera nuestro programa.

Por ejemplo, nuestra aplicación trabaja con una interfaz llamada **SistemaPago** que posee el método **void pagar(double monto);**

Sin embargo, tenemos una clase externa llamada **PasarelaExterna** que realiza pagos mediante otro método **void realizarCobro(double cantidad);**

Aunque ambas clases realizan prácticamente la misma función, el programa no puede utilizar directamente **PasarelaExterna** como un **SistemaPago** porque sus interfaces son diferentes.

Modificar la clase externa puede no ser posible o no ser conveniente, especialmente si pertenece a una librería externa.

Solución

En el patrón se crea una clase intermedia llamada “**Adapter**”, cuya función es hacer compatibles ambas interfaces.

En este ejemplo se crea la clase **PasarelaAdapter**

Esta clase implementa la interfaz **SistemaPago**, que es la interfaz que espera nuestra aplicación.

Internamente, el Adapter contiene una instancia de **PasarelaExterna**.

Cuando el programa llama a **pagar(monto)**;

El Adapter transforma esa llamada en **pasarelaExterna.realizarCobro(monto)**;

De esta manera, el código principal puede trabajar con la interfaz **SistemaPago** sin preocuparse por cómo funciona internamente la pasarela externa.

El Adapter actúa como un traductor entre dos interfaces incompatibles.

Consecuencias

Ventajas

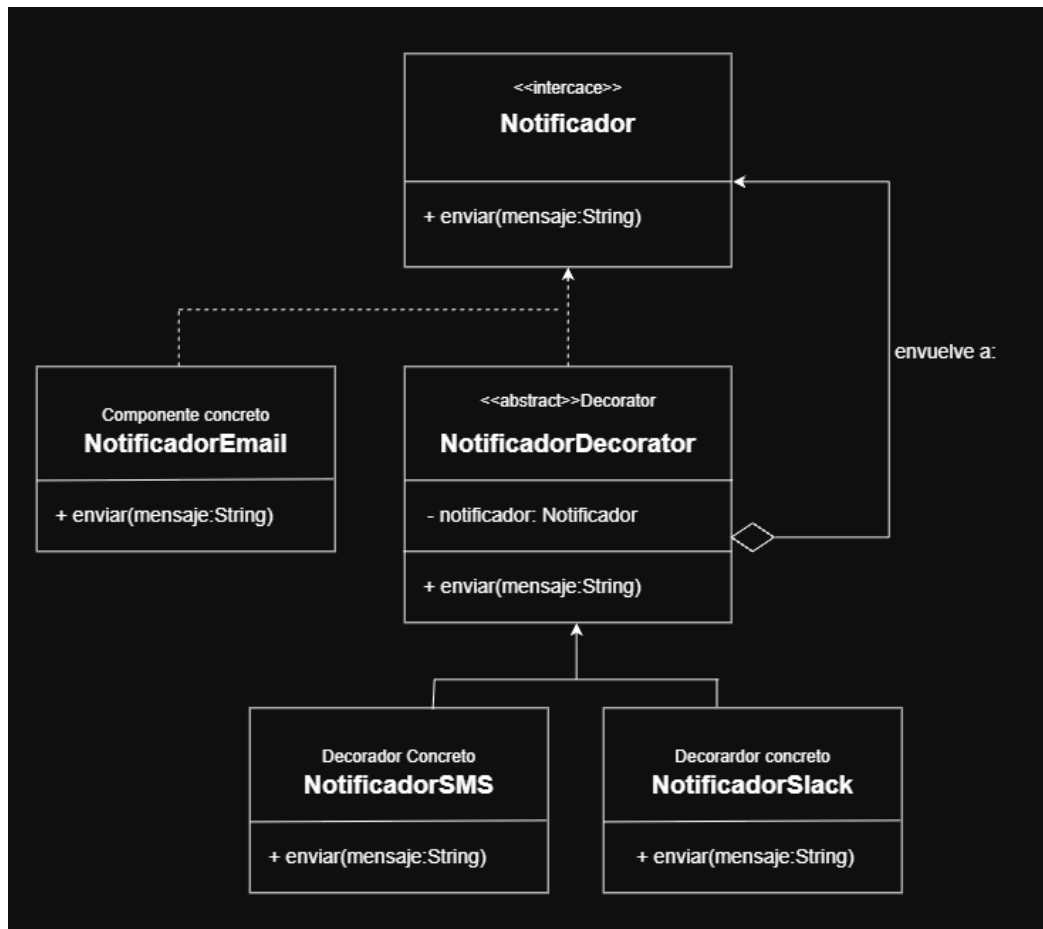
- Permite reutilizar clases existentes.
- Evita modificar clases externas.
- Reduce el acoplamiento.
- Facilita integrar sistemas externos.
- Separa la lógica de adaptación.
- Aumenta la flexibilidad.

Desventajas

- Aumenta la cantidad de clases.
- Puede agregar complejidad.
- Puede haber muchos adapters.
- El Adapter debe conocer ambas interfaces.

Decorator

UML



Problema

Queremos agregar funcionalidades a un objeto y poder combinarlas como queramos, pero las opciones tradicionales fallan:

- **Con herencia** hay una explosión de subclases. Para combinar email, SMS y Slack necesitarías EmailSMS, EmailSlack, EmailSMSSlack... y cada canal nuevo duplica las combinaciones. Además, la herencia es estática: se decide al compilar, no mientras corre el programa.
- **Modificar la clase original** (meter if (sms), if (slack) dentro de NotificadorEmail) la vuelve cada vez más grande y frágil, y rompe el principio Open/Closed (abierto a extensión, cerrado a modificación).

Solución

El Decorator **envuelve** al objeto original con otro objeto que implementa la misma interfaz. El envoltorio delega en el objeto envuelto y agrega su propio comportamiento.

- **Sin tocar la clase original:** NotificadorEmail no cambia nunca. Cada funcionalidad nueva es una clase nueva (NotificadorSMS, NotificadorSlack).

- **Dinámico:** las capas se apilan en tiempo de ejecución, como en `new NotificadorSlack(new NotificadorSMS(new NotificadorEmail()))`.
- **Transparente para el cliente:** todo es un Notificador, así que quien lo usa llama a `enviar()` sin saber cuántas capas tiene.

Ventajas

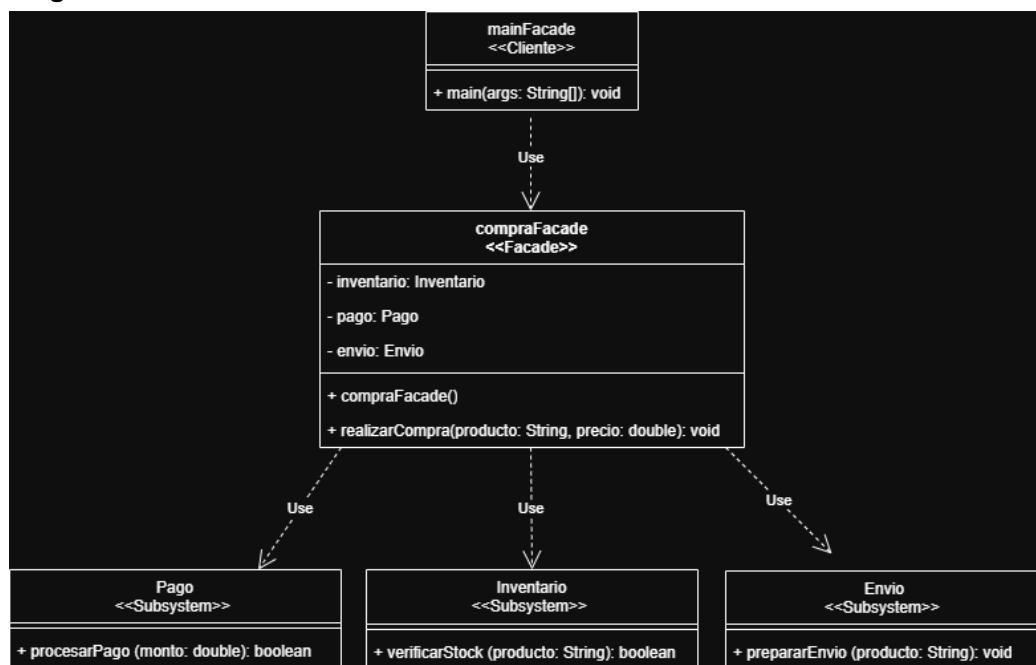
- Es más flexible que la herencia: se combinan funcionalidades en ejecución.
- Cumple Open/Closed: se extiende sin modificar código existente.
- Cada decorator tiene una sola responsabilidad (un canal), lo que facilita el mantenimiento.
- Evita clases gigantes llenas de condicionales.

Desventajas

- Genera muchas clases pequeñas y parecidas.
- Armar el objeto es más verboso (varios `new` anidados) y cuesta más leer.
- El orden de las capas puede importar.
- El objeto decorado no es el original: comparaciones con `==` o `instanceof` pueden dar resultados inesperados.
- Es difícil quitar un decorator del medio de la cadena.

Facade

1. Código Java funcionando
2. Diagrama UML



3. Problema - Solución - Consecuencias

a. Problema:

Cuando una aplicación crece, el cliente (el código que usa los objetos) tiene que interactuar directamente con muchas clases complejas y acopladas al mismo tiempo (en este ejemplo: verificar stock, cobrar, coordinar envíos). Esto hace que el código

del cliente sea difícil de leer, mantener y que cualquier cambio en los subsistemas rompa al cliente.

b. Solución:

Se coloca una clase intermediaria (la Fachada o compraFacade) que funciona como una "ventanilla única". En lugar de que el cliente hable con tres clases distintas, ahora habla únicamente con la fachada, y es la fachada la que se encarga de orquestar y ordenar a los subsistemas por detrás.

c. Consecuencias

- **Ventajas:**

- **Código más limpio y fácil de usar:** El cliente queda súper simple (en lugar de 10 líneas de código con lógica dispersa, solo llama a un método como realizarCompra).
- **Menor acoplamiento:** Si mañana cambias cómo funciona la clase Pago, solo modificas la fachada o el subsistema, pero el cliente (Main) ni se entera y no hay que tocarlo.

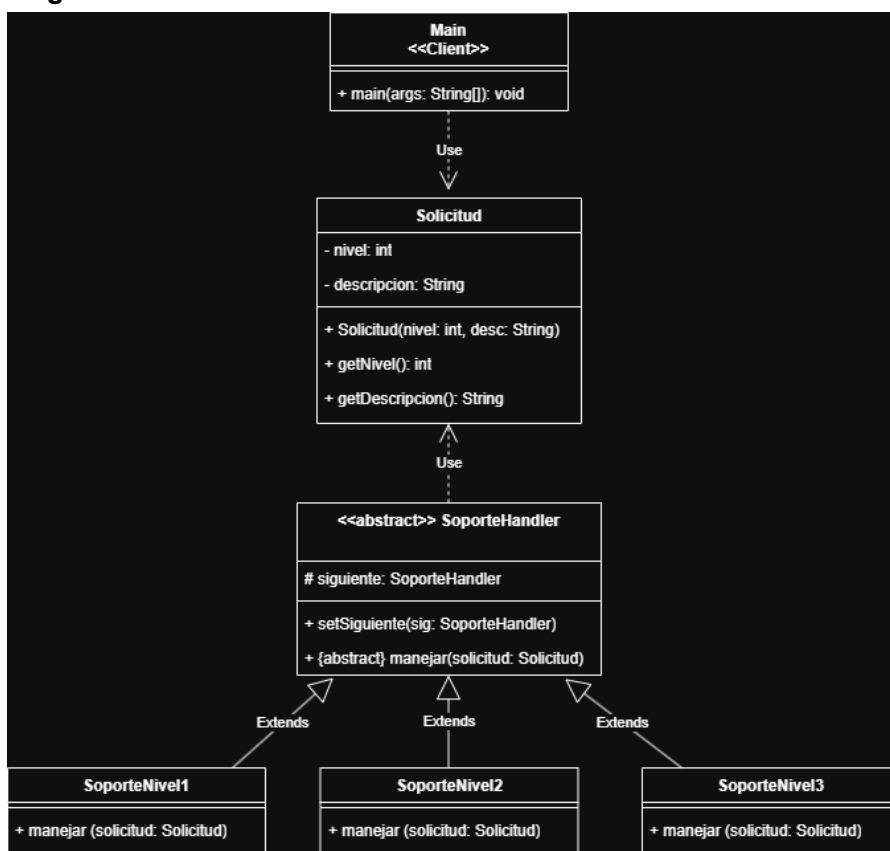
- **Desventajas:**

- Si la fachada crece demasiado y empieza a hacer de todo, puede convertirse en un objeto gigantesco y difícil de mantener (se le llama "objeto todopoderoso").

Behavioral (Conductuales):

Chain of Responsibility

1. Código Java funcionando
2. Diagrama UML



3. Problema - Solución - Consecuencia

a. Problema:

Cuando tienes una petición o solicitud que puede ser procesada por diferentes objetos (por ejemplo, distintos niveles de soporte técnico), si usas un enfoque tradicional con condicionales gigantes (if/else if), el código se vuelve rígido, difícil de mantener y viola el principio de diseño de "responsabilidad única". Cada vez que agregues un nuevo nivel de soporte, tendrías que modificar una clase central rompiendo código existente.

b. Solución:

Implementar el patrón Chain of Responsibility. Consiste en crear una cadena de objetos receptores (los manejadores) donde cada uno tiene una referencia al siguiente. Cuando llega una solicitud, el primer objeto evalúa si puede resolverla; si no puede, se la pasa automáticamente al siguiente eslabón de la cadena sin que el cliente tenga que saber quién la terminó resolviendo.

c. Consecuencia:

- **Ventajas:**

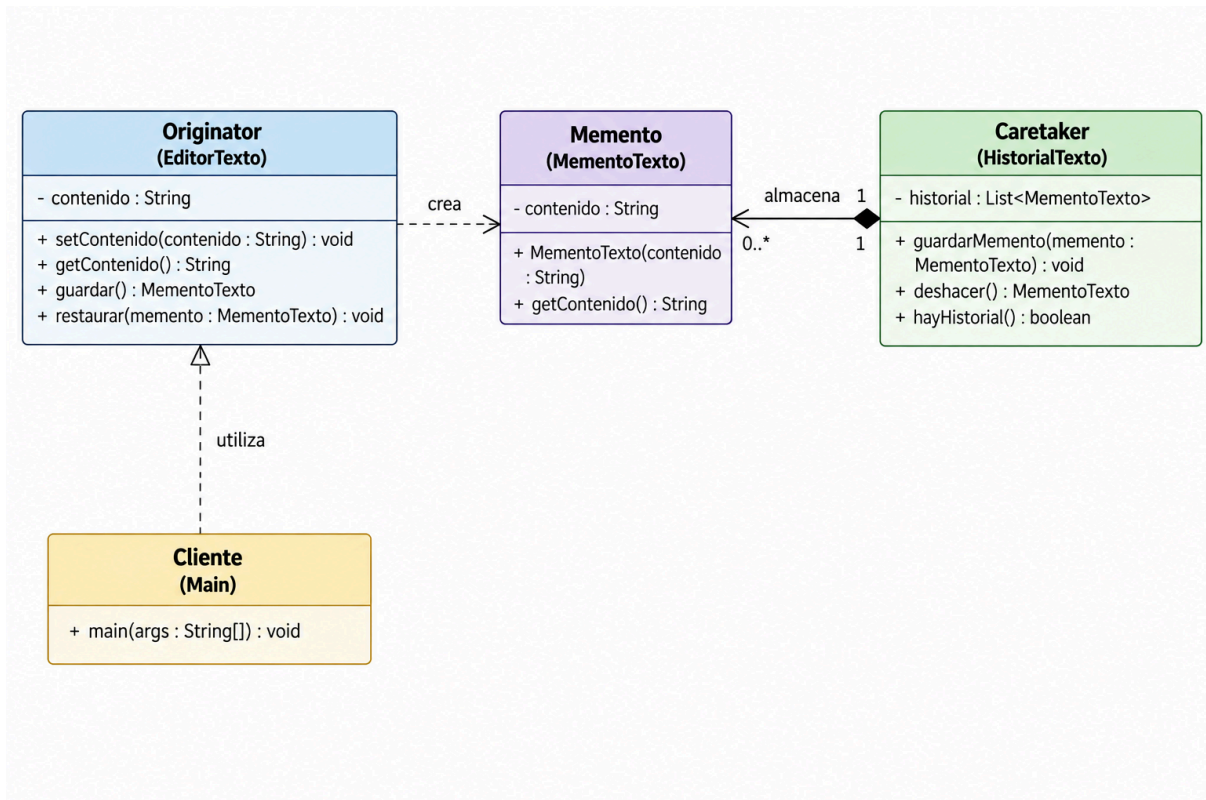
- **Desacoplamiento:** El objeto que envía la solicitud (el cliente) no necesita saber qué objeto exacto la va a resolver.
- **Flexibilidad:** Puedes agregar, quitar o reordenar los eslabones de la cadena fácilmente modificando el Main (o mediante configuración), sin tocar el código interno de los demás manejadores.
- **Principio de responsabilidad única:** Cada clase manejadora se encarga exclusivamente de su propia lógica de decisión.

- **Desventajas:**

- **Sin garantía de atención:** Si la cadena está mal armada o ningún objeto maneja la solicitud, esta puede llegar al final de la cadena y perderse o no resolverse.
- **Rendimiento:** Si la cadena es muy larga, puede haber un pequeño costo de procesamiento al recorrer los eslabones hasta encontrar al responsable.

Memento

Diagrama UML



Problema

El patrón Memento se utiliza cuando necesitamos guardar el estado de un objeto para poder restaurarlo posteriormente. Un ejemplo típico es un editor de texto con la función **deshacer**.

Solución

El patrón **Memento** permite guardar una copia del estado interno de un objeto sin exponer directamente sus atributos.

En este ejemplo participan:

- **EditorTexto**: es el **Originator**. Contiene el estado actual del texto.
- **MementoEditor**: es el **Memento**. Guarda una copia del contenido del editor.
- **Historial**: es el **Caretaker**. Guarda los distintos Mementos.
- **MainMemento**: utiliza las clases y demuestra cómo guardar y restaurar estados.

Cuando queremos guardar el estado actual:

```
editor.guardar();
```

se crea un nuevo **MementoEditor**.

El historial almacena ese Memento:

```
historial.guardar(editor.guardar());
```

Luego, si queremos volver a un estado anterior:

```
editor.restaurar(historial.deshacer());
```

El objeto **EditorTexto** recupera el contenido que tenía anteriormente.

Consecuencias

Ventajas

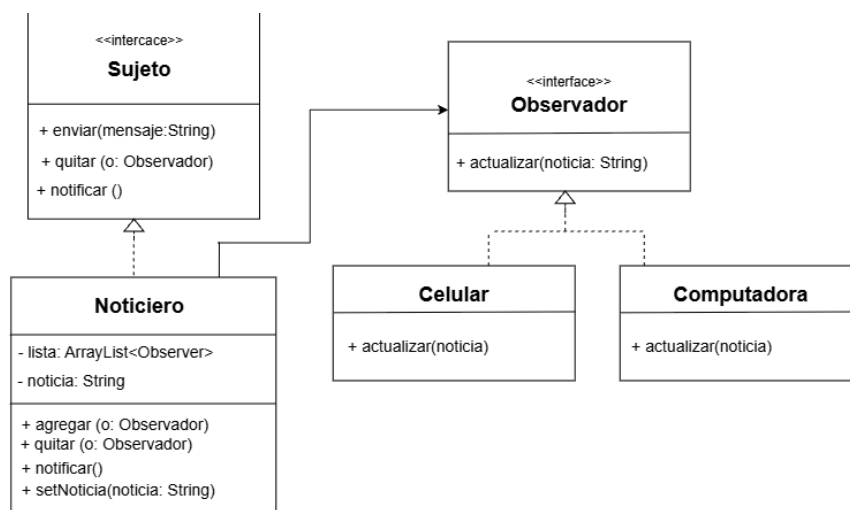
- Permite implementar deshacer:
 - Mantiene el encapsulamiento:
 - Separa las responsabilidades:
 - Facilita guardar diferentes versiones:
 - Evita que el historial conozca los detalles internos
-

Desventajas

- Puede consumir mucha memoria
- Puede generar muchos objetos
- La administración del historial puede ser compleja
- No siempre es necesario

Observer

UML



Problema

Hay objetos que dependen del estado de otro objeto y necesitan enterarse cuando cambia. En nuestro ejemplo, el **Noticiero** tiene una noticia, y el **Celular** y la **Computadora** quieren recibirla apenas cambia.

Sin el patrón hay dos formas de resolverlo, y las dos tienen problemas:

- Preguntar todo el tiempo: cada objeto consulta al noticiero cada tanto si hay algo nuevo. Se gasta trabajo de más y puede que se enteren tarde.
- Que el noticiero los conozca a todos: el noticiero llama directamente a Celular y a Computadora. Queda muy pegado a ellos, y para agregar un objeto nuevo hay que modificar su código.

Solución

El patrón Observer separa los roles en dos:

- El Sujeto (Noticiero) guarda una lista de observadores. Cuando cambia su estado, recorre la lista y le avisa a cada uno llamando a actualizar().
- Los Observadores (Celular, Computadora) se registran con agregar() y se dan de baja con quitar().

El sujeto no sabe qué tipo de objetos lo escuchan, solo sabe que implementan la interfaz Observador. Así los avisos son automáticos y se pueden sumar o sacar observadores sin tocar el código del sujeto.

Ventajas

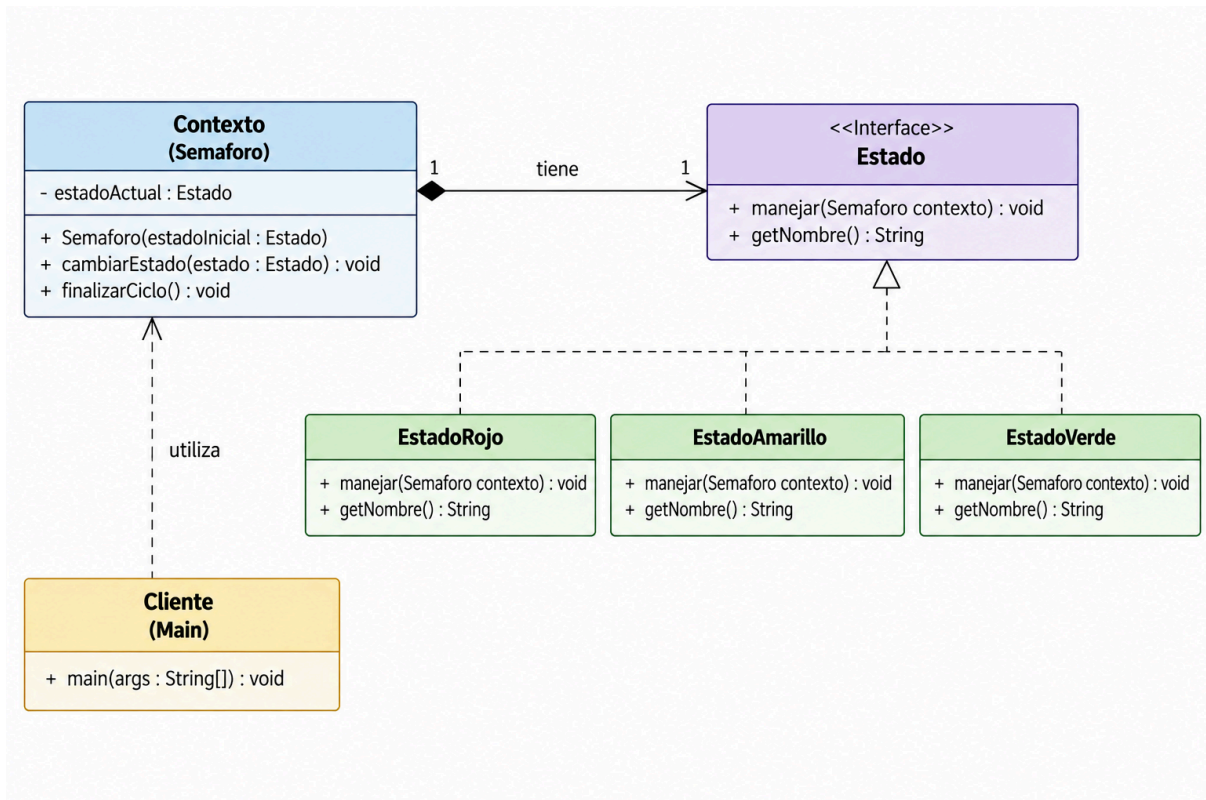
- Hay bajo acoplamiento: el sujeto solo conoce la interfaz Observador, no las clases concretas.
- Se pueden agregar o quitar observadores en cualquier momento, incluso mientras el programa corre.
- Las notificaciones son automáticas, sin necesidad de estar consultando.
- Un mismo sujeto puede avisar a muchos observadores distintos, cada uno reaccionando a su manera.

Desventajas

- Si hay muchos observadores, notificar a todos puede volverse lento.
- Los observadores se notifican en un orden que no se controla, así que no conviene depender de ese orden.
- Un observador que no se quita de la lista sigue recibiendo avisos aunque ya no se use (puede generar errores o gastar memoria).
- Es difícil seguir el flujo del programa, porque un solo cambio dispara muchas acciones que no se ven a simple vista.

State

Diagrama UML



Problema

El patrón **State** se utiliza cuando un objeto puede comportarse de manera diferente dependiendo del estado en el que se encuentra.

Por ejemplo, un reproductor de música puede encontrarse en distintos estados:

- Reproduciendo
- Pausado
- Detenido

Solución

El patrón **State** separa el comportamiento de cada estado en una clase diferente.

Todas las clases de estado implementan una misma interfaz.

En este ejemplo participan:

- **EstadoReproductor**: interfaz que define el comportamiento de los estados.
- **EstadoReproduciendo**: representa el estado reproduciendo.
- **EstadoPausado**: representa el estado pausado.
- **EstadoDetenido**: representa el estado detenido.
- **Reproductor**: es el contexto y mantiene el estado actual.
- **MainState**: demuestra el funcionamiento del patrón.

El **Reproductor** no necesita contener grandes bloques de condicionales.

Simplemente delega el comportamiento al estado actual:

estado.actualizarEstado(this);

Cada estado decide qué debe hacer y a qué nuevo estado debe cambiar el reproductor.

Ventajas

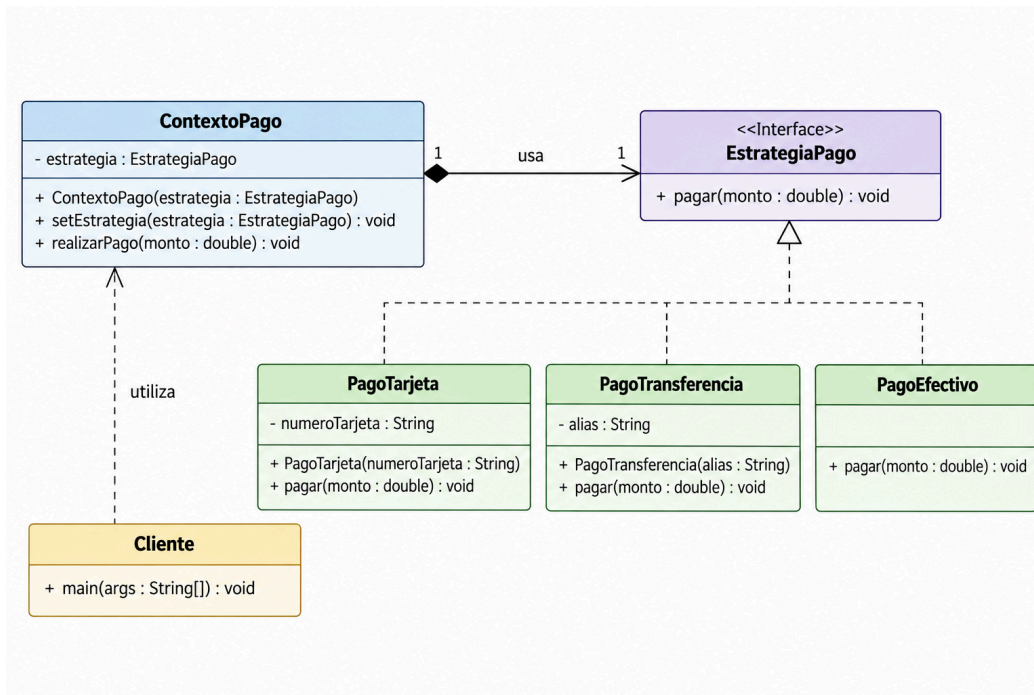
- Evita grandes bloques de condicionales
 - Organiza mejor los estados:
 - Facilita agregar nuevos estados
 - Permite cambiar el comportamiento en tiempo de ejecución
 - Mejora el mantenimiento
 - Separa el estado del contexto
-

Desventajas

- Aumenta la cantidad de clases
- Puede ser innecesario para pocos estados
- Las transiciones pueden volverse complejas
- Los estados deben conocer el contexto

Strategy

Diagrama UML



Problema

El patrón **Strategy** se utiliza cuando un objeto puede realizar una misma tarea de distintas maneras y queremos poder cambiar el algoritmo o comportamiento sin modificar la clase principal.

Por ejemplo, un sistema de pagos puede permitir diferentes formas de pago:

- Tarjeta de crédito
- PayPal
- Transferencia bancaria

Sin Strategy, podríamos terminar utilizando muchos if, else o switch dentro de una misma clase para decidir qué forma de pago usar.

Esto hace que el código sea más difícil de mantener y ampliar.

Solución

El patrón Strategy separa cada algoritmo o comportamiento en una clase diferente.

Todas estas clases implementan una misma interfaz.

En este ejemplo:

- **EstrategiaPago** define la operación `pagar()`.
- **PagoTarjeta** implementa el pago con tarjeta.
- **PagoPayPal** implementa el pago con PayPal.

- **ContextoPago** utiliza una estrategia de pago.
- **MainStrategy** selecciona qué estrategia utilizar.

La clase **ContextoPago** no necesita saber cómo funciona cada método de pago.

Simplemente ejecuta **estrategiaPago.pagar(monto);**

De esta manera, el comportamiento puede cambiarse en tiempo de ejecución.

Consecuencias

Ventajas

- Evita muchos condicionales
- Permite cambiar el comportamiento en tiempo de ejecución
- Facilita agregar nuevas estrategias
- Separa los algoritmos de la clase principal
- Mejora el mantenimiento
- Cumple con el principio abierto/cerrado

Desventajas

- Aumenta la cantidad de clases
- El cliente debe conocer las estrategias disponibles
- Puede ser innecesario para pocos comportamientos
- Puede haber muchas clases pequeñas